RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

CS4104 – COMPUTER GRAPHICS AND VIRTUAL REALITY

PROJECT / CASE STUDY REPORT

TITLE OF THE SELECTED CASE STUDY

CASE STUDY 3: RETAIL AND REMOTE COMMERCE

Submitted by

Munnaluri Rishikesh

USN: 1RVU23CSE291

Under the guidance of

Dr./Prof. _____

School of Computer Science and Engineering

RV University, Bengaluru

Academic Year: 2026–2027

RV UNIVERSITY

School of Computer Science and Engineering

CERTIFICATE

This is to certify that the CS4104 – Computer Graphics and Virtual Reality case study report titled “Case Study 3: Retail and Remote Commerce” has been prepared and submitted by Munnaluri Rishikesh (USN: 1RVU23CSE291) of the School of Computer Science and Engineering, RV University, Bengaluru, during the academic year 2026–2027. The report has been prepared in accordance with the course requirements and instructions provided.

Dr./Prof. _____

Faculty Guide

ABSTRACT

An interior design startup aims to present furniture inside a customer's home through an augmented or virtual reality (AR/VR) experience. Convincing previews require surfaces that react coherently to light, detailed material appearance without excessive geometry, and correct mapping from interaction coordinates to display pixels. This case study examines three related Computer Graphics and Virtual Reality (CGVR) questions in the context of a red velvet chair and a brown leather sofa. A Unity 6000.3.0f1 desktop showroom demonstrates the underlying graphics operations: a custom Phong-style shader separates ambient, diffuse and specular terms; a procedurally generated leather albedo and normal texture modifies the sofa appearance without modifying mesh geometry; and a viewport panel converts a point in a normalized logical window to an illustrative smartphone pixel rectangle. The worked mapping (0.25, −0.30) → (657.5, 708.0) pixels is derived step by step. The report also describes an implementation workflow, camera interaction, limitations, and optimization choices related to rendering budgets and mobile/WebXR constraints. Two authentic Unity Play-mode screenshots from the earlier working build provide visual evidence of a running showroom. The application remains a desktop educational simulation, not a device-tracked WebXR deployment; runtime measurements and the most recent presentation changes have not been independently verified in these captures. The design shows how core mathematical graphics models support practical retail visualization and explains the additional systems needed for production AR.

KEYWORDS

Computer graphics; Unity; WebXR concept; Phong illumination; texture mapping; normal mapping; viewport transformation; interactive furniture visualization.

TABLE OF CONTENTS

1. Introduction
2. Problem Statement and Objectives
3. Case Scenario and Requirements
4. Relevant CG/VR Concepts
5. Technical Analysis and Proposed Solution
6. System Design and Workflow
7. Performance, Optimization and Limitations
8. Applications and Discussion / Practical Evidence
9. Conclusion and Future Scope

References and Appendix

1. INTRODUCTION

1.1 Background and motivation

Furniture shopping traditionally requires showroom visits or interpretation of catalog photographs. These sources provide useful appearance information but can fail to communicate how a chair or sofa fits in a particular room, how its fabric responds to the available lighting, or how a chosen model appears from a customer's viewpoint. Retail AR and VR attempt to reduce this uncertainty by rendering three-dimensional products in context. The technical challenge is not limited to building a mesh: rendering, materials and screen-space interactions must produce a consistent visual presentation [1], [2].

1.2 Why this case study matters

The selected RV University scenario concerns an interior design startup planning a WebXR furniture-preview application. A virtual red velvet chair must remain visually convincing in a sunlit home, a leather sofa should show fine surface detail without excessive triangles, and touch or gaze interaction must map to the correct displayed region. These are representative problems in illumination modelling, appearance modelling and geometric transformation. They allow direct links between classroom equations and a usable demonstration [1].

1.3 Purpose and scope of the prototype

A small desktop Unity scene was chosen as a controlled teaching environment. It contains a red chair, brown sofa, floor, walls, window, simple props, adjustable illumination and on-screen controls. The model produces its scene elements procedurally during Play mode. It is used to observe three graphics principles and compare settings, not to assert photorealism or full hardware AR functionality. Its source code, shader and scene definition are available in the project repository [2].

2. PROBLEM STATEMENT AND OBJECTIVES

Problem statement: design and explain a rendering workflow that makes furniture objects readable under differing illumination, adds convincing material characteristics without unnecessarily increasing polygon count, and accurately maps logical interaction coordinates to a display viewport, while keeping the demonstrator responsive.

- Model the ambient, diffuse and specular lighting contributions on a simulated red velvet chair.

- Apply UV colour texture and normal-map detail to a brown sofa and explain why geometry stays unchanged.
- Derive a reproducible window-to-viewport mapping and relate it to phone and Unity Game-view coordinates.
- Present a practical Unity workflow with camera controls, material switches and intelligible visual evidence.
- Discuss rendering cost, frame-time budgets and limitations without inventing benchmark results.

3. CASE SCENARIO AND REQUIREMENTS

3.1 Scenario and stakeholders

The proposed customer journey is a furniture preview in a domestic environment. A user selects a chair or sofa, explores the object from several angles, examines fabric characteristics and interacts with the virtual content through a screen. The startup needs assets that are recognizable, lightweight and consistent across display sizes. Relevant stakeholders include a customer comparing furniture, a designer preparing models, a graphics developer implementing material and camera logic, and a product team deciding whether the approach can eventually run on phone-based AR or VR hardware.

3.2 Required visual and interaction behaviour

Requirement	Purpose and case-specific solution
Natural chair illumination	Separate ambient base colour, Lambertian diffuse response and controlled specular reflection.
Detailed leather surface	Sample a UV albedo map and tangent-space normal map; preserve original mesh topology.
Accurate screen coordinates	Convert a point from a logical interval to a phone pixel rectangle and distinguish GUI coordinates.
User exploration	Orbit, zoom and reset a camera; focus chair or sofa; expose graphics modes.
Readable demonstration	Show the room and controls at a usable landscape aspect ratio.

3.3 Functional and non-functional requirements

Functional requirements cover furniture display, selectable lighting modes, independent texture and bump toggles, coordinate sliders, an inset-view control, and keyboard/mouse interaction. Non-functional goals cover readability, predictable behaviour, minimal dependencies, reasonable rendering cost and reproducibility on the specified Unity version. A production WebXR implementation would additionally require device tracking, placement,

occlusion, camera permissions, performance profiling and broader accessibility validation; those systems are outside this desktop prototype [2], [3].

3.4 Boundaries of validation

A working Play-mode build of the earlier Unity scene was observed and captured in original user-supplied screenshots. Those images show actual scene rendering and the graphical panels. Source-level presentation changes were subsequently committed; this report does not treat the earlier screenshots as proof that the newest changes were run. Likewise, proposed AR interactions are discussed as engineering extensions rather than implemented device features.

4. RELEVANT CG/VR CONCEPTS

4.1 The rendering pipeline and illumination

A triangle mesh is transformed from local coordinates to world and view space, projected, clipped, rasterized and shaded. The rasterizer interpolates vertex attributes such as normals and UV coordinates. Fragment shading uses this information to produce surface colour. The Unity demonstration implements a compact forward shading model in ShaderLab/HLSL, making the light contributions observable rather than hiding them inside an unrelated material preset [2], [3].

4.2 Ambient, diffuse and specular reflection

Ambient illumination supplies a baseline reflected contribution even where direct light is weak. In the simplified model it is proportional to the surface albedo and an ambient strength coefficient. Diffuse illumination, associated with matte surfaces such as velvet, depends on the angle between surface normal N and light direction L. The Lambertian term is max(N·L, 0), ensuring no negative intensity on back-facing surfaces. Specular illumination depends on the view direction V and ideal reflected direction R, with an exponent n controlling highlight sharpness [4].

$$I = I_a k_a + I_p \left[k_d \max(N \cdot L, 0) + k_s \max(R \cdot V, 0)^n \right]$$

For the red chair, a saturated red albedo and relatively low specular strength convey a fabric-like response. The sofa uses a brown textured albedo and a more visible highlight to suggest leather. The coefficient choices are illustrative, not calibrated measurements of actual commercial furniture. In outdoor or sunlit rooms, a production solution would need environment lighting, shadow consistency, tone mapping and real-camera colour matching.

4.3 Difference from physically based rendering (PBR)

Phong provides understandable components and low conceptual overhead. PBR extends material reasoning using parameters such as roughness, metallic response and energy-conserving reflectance, and is generally preferred for production-grade consistency across light sources. The lab intentionally exposes Phong's ambient/diffuse/specular factors because the faculty question asks students to explain those effects explicitly. A PBR retail renderer would be a reasonable next-stage improvement, not a feature claimed for this prototype [3], [4].

4.4 Texture mapping and UV coordinates

A geometric sofa model provides surfaces but does not inherently define fine leather grain. Texture mapping attaches a two-dimensional image to a polygon mesh by storing UV coordinates for its vertices. After interpolation across the rendered triangle, the fragment shader samples the albedo texture at each (u, v) location. Multiplication with a material tint preserves design control over the base colour without changing the object's vertices [3].

$$\text{Albedo}(u,v) = \text{FurnitureTint} \times \text{TextureSample}(\text{AlbedoTexture}, u, v)$$

4.5 Bump and normal mapping

Bump mapping approximates small variations in surface orientation; normal mapping stores the resulting orientations more directly. A tangent-space normal map encodes x, y and z normal components in RGB values. Decoding maps the stored [0,1] values to [−1,1], and a tangent-bitangent-normal (TBN) basis transforms that local normal into world space for shading. The shader then uses this perturbed normal in both diffuse and specular calculations. The apparent small pores and creases can influence highlights although the polygon boundaries remain unchanged [2], [3].

$$n_{tangent} = \text{normalize}(2 \times \text{RGB_normal_map}(u,v) - 1)$$

$$n_{world} = \text{normalize}(T \times n_x + B \times n_y + N \times n_z)$$

4.6 Trade-offs and technical limitations

A normal map creates the impression of microscopic detail economically, but it cannot change silhouettes, cast truly displaced pore shadows or solve inaccurate UV mapping. It also adds texture samples and memory traffic to the fragment stage. The current Unity script creates the colour and normal textures procedurally at runtime using deterministic calculations, so no external material downloads are needed. The updated source specifies two 256 × 256 RGBA textures and exposes separate on/off switches for educational comparison.

Texture-mapping quality depends on sampling, lighting, camera angle and normal-map strength.

4.7 The distinction relevant to assessment

Texture mapping changes the surface colour pattern. Bump/normal mapping changes the shading direction. Increasing tessellation changes the actual geometry. These are three different mechanisms, which is why the current lab keeps the sofa mesh fixed while toggling colour and bump independently. The discussion answers the scenario's specific requirement: improved leather appearance without increasing mesh polygon count.

4.8 Window-to-viewport transformation

A window is a logical region where coordinates have meaning before being displayed. A viewport is the rectangular destination in a physical or logical display. Once a point is expressed in the window's coordinate system, normalized interpolation determines its destination coordinates. This operation is independent of the visual scene but essential to matching a pointer, touch point or projected marker to the correct region of a phone or headset panel.

4.9 General equations

$$x_v = x_{vmin} + (x_w - x_{wmin}) \times (x_{vmax} - x_{vmin})/(x_{wmax} - x_{wmin})$$

$$y_v = y_{vmin} + (y_w - y_{wmin}) \times (y_{vmax} - y_{vmin})/(y_{wmax} - y_{wmin})$$

Both source intervals must have nonzero width. A bottom-left origin convention is convenient for conventional graphics derivations, while some GUI systems position controls from the top-left corner. To render the same logical location in an interface, the vertical coordinate may need to be reversed. A correct implementation must state the origin and avoid mixing screen pixels, normalized viewport values and 3D world coordinates [1], [3].

4.10 Connection to 3D interaction

For a virtual chair on a phone, object points pass through a model matrix, a view matrix and a projection matrix before perspective division and viewport mapping. Touch input follows a related inverse route for picking: screen position becomes a ray or world-space query after camera and projection information are considered. The case-study prototype illustrates the explicit 2D viewport mapping and camera-rectangle changes. It does not implement full AR plane tracking or headset gaze selection.

4.11 Colour models in the pipeline

The chair and sofa are represented by RGB colours and textures. Digital output depends on display transfer functions and scene colour-space handling; a production renderer might use linear-light calculations and sRGB textures for perceptually correct compositing. The teaching shader illustrates geometry-dependent contributions clearly but should not be taken as a colour-calibrated matching engine for real fabrics. This limitation is relevant when explaining why a red chair may look different on different screens.

5. TECHNICAL ANALYSIS AND PROPOSED SOLUTION

5.1 Question 1 — Illumination and colour models

Faculty question: When a virtual red velvet chair is placed in a brightly sunlit room, which illumination models must the engine compute to blend it naturally with real-world lighting? The solution must combine stable base visibility, directional lighting response and restrained highlights, while keeping the chair's red material identifiable.

5.1.1 Ambient term

The ambient term prevents areas without direct illumination from appearing entirely black. Conceptually, C_ambient = albedo × ambientStrength. A modest ambient level improves readability in the educational showroom; for real AR, that coefficient should be informed by estimated environment light rather than selected arbitrarily.

5.1.2 Diffuse term

A fabric surface should be brighter where it faces the window or simulated sun and darker where it turns away. The dot product max(N·L,0) gives this angular effect. Because velvet is visually more matte than polished leather, diffuse response provides much of the chair's apparent volume. The custom Phong shader uses per-fragment normals and directional-light colour to compute this contribution.

5.1.3 Specular term

Highlights occur when the viewer is near the ideal reflected-light direction. The model raises max(R·V,0) to a shininess exponent. A smaller coefficient is appropriate for the simplified red fabric, while the leather sofa receives stronger, more visible specular treatment. Users can compare ambient-only, ambient-plus-diffuse and full ambient/diffuse/specular modes using keyboard keys 1, 2 and 3 [2].

5.1.4 Practical application and evaluation

The Unity controller builds a room and assigns shader materials to the furniture. A directional light and simulated ambient environment support the demonstration; orbiting the camera changes the view vector for specular reflection. The reported outcome is pedagogical: the three components can be enabled to illustrate how light affects appearance. A production AR engine would additionally acquire light-estimation and shadow/environment information from the device, then address exposure, occlusion, colour calibration and latency.

5.2 Question 2 — Texture mapping and bump mapping

Faculty question: How can a leather sofa appear detailed and realistic without increasing mesh polygon count? The recommended solution retains a coarse sofa mesh and changes material sampling and per-pixel normals in the fragment stage.

5.2.1 Applying the albedo map

Each mesh vertex stores a UV coordinate. The rasterizer interpolates it for each visible fragment. Sampling the brown leather albedo map adds variations associated with surface grain. The model does not add vertices when the pattern changes. In the current Unity project, a 256 × 256 texture is generated procedurally using a deterministic height-like pattern and colour modulation; the GUI and T shortcut can switch the colour texture on or off [2].

5.2.2 Applying the normal map

The same procedural pattern is differentiated numerically across neighbouring texture pixels to construct a normal texture. The custom fragment shader decodes the RGB normal to a signed vector, multiplies its tangent-space x and y components by a configurable bump strength, converts it with the local tangent basis, and uses the result for diffuse/specular terms. The B key or material control independently enables and disables the bump effect. The mesh topology remains unchanged.

5.2.3 Why it is efficient and what it cannot do

Normal mapping can approximate fine leather irregularity with two-dimensional samples rather than additional triangles and geometry processing. The main cost moves into fragment shader operations and texture sampling; its savings are useful when the target includes limited mobile GPUs. Nonetheless, significant holes, folds or silhouette changes require actual mesh geometry, and high-quality material appearance may require physically based shading and calibrated texture capture. Normal mapping does not literally displace the surface.

5.2.4 Evidence and student observation

The earlier Play-mode screenshot shows the leather sofa in the showroom, and the visible interface presents texture and bump controls. That evidence supports the existence of the demonstration UI, not a quantitative proof of the visual difference between both toggle states. The source code contains independent _UseTexture and _UseBump shader parameters; before-and-after renders are suggested as additional evidence if the faculty requests explicit comparisons.

5.3 Question 3 — Mapping user interaction to the viewport

Faculty question: How does window-to-viewport transformation accurately map application interaction space to a smartphone or VR headset display? The solution normalizes the logical point relative to the source rectangle and scales/translates that position into the destination pixel rectangle.

5.3.1 Values supplied in the Unity demonstration

Symbol / variable	Example value
Logical x and y ranges	−1 to +1 on both axes
Selected logical point	(xw, yw) = (0.25, −0.30)
Pixel viewport bounds	x = 70..1010; y = 120..1800
Coordinate origin	Bottom-left for the mathematical example

5.3.2 Worked calculation

$$x_v = 70 + [(0.25 - (-1))/(1 - (-1))] \times (1010 - 70)$$

$$= 70 + (1.25/2) \times 940 = 657.5 \text{ pixels}$$

$$y_v = 120 + [(-0.30 - (-1))/(1 - (-1))] \times (1800 - 120)$$

$$= 120 + (0.70/2) \times 1680 = 708.0 \text{ pixels}$$

Final mapped point = (657.5, 708.0) pixels

5.3.3 Unity viewport and interface interpretation

The “Window to Viewport” panel displays the mapped phone-example point and allows logical X and Y to change. Unity’s own camera viewport is a separate rectangle described by Camera.rect and Camera.pixelRect. An Inset/Fill option changes the rendered camera region. Because Unity GUI elements use a top-left origin, the drawn marker must flip its Y placement relative to a bottom-left mathematical coordinate. A headset introduces additional per-eye viewports and projection matrices, but the same coordinate-normalization principle remains applicable [2], [3].

6. SYSTEM DESIGN AND WORKFLOW

6.1 Layered architectural view

The application separates scene creation, viewer interaction, material shading and screen-space coordinate conversion. Keeping the components explicit supports debugging and faculty explanation: a camera can be moved without modifying furniture texture geometry, and the viewport marker can be changed without altering the shaded sofa.

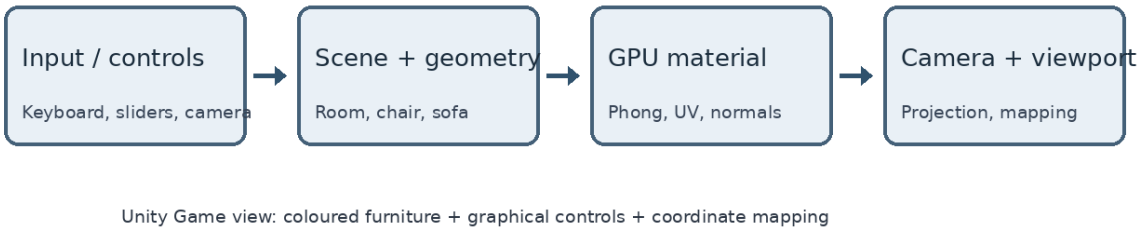


Figure 1. Conceptual workflow of the Unity CGVR educational visualizer (architecture illustration, not a runtime screenshot).

6.2 Rendering path

At startup, the controller creates primitive furniture geometry, assigns a custom shader to the chair and sofa, generates procedural leather textures, creates a directional light and constructs a camera. Each frame the input controller updates lighting mode, selected object, view orbit and material switches. Vertex processing transforms the mesh, fragment processing evaluates texture/normal samples and light terms, and the camera sends the result to the current Game viewport. GUI controls display the relevant parameters [2].

6.3 Core project components and reproducibility

Unity project component	Implementation role
CIE3_Retail_Furniture_Visualizer.unity	Saved launch scene; controller object references shader.
RetailARCaseStudyDemo.cs	Procedural furniture, textures, lighting, camera, controls and viewport mapping.
PhongFurniture.shader	Per-fragment ambient, diffuse, specular, albedo and normal-map shading.
CIE3SceneBuilder.cs	Editor menu for opening saved scene and optional visualizer setup.
CIE3Validation.cs	Source-level/Editor checks for scene, shader and mapping mathematics.

6.4 Procedure for an assessor to reproduce the demonstration

- Install/open Unity 6000.3.0f1 using the Built-in Render Pipeline.
- Open the repository's CASE STUDY CIE3/Unity_Retail_AR_Demo folder in Unity Hub (the directory containing Assets, Packages and ProjectSettings).
- Open Assets/Scenes/CIE3_Retail_Furniture_Visualizer.unity or the CGVR CIE3 menu item.
- Press Play and select Game. The room and furniture are generated at runtime.
- Press R to reset the camera, then compare keys 1, 2, 3 for Phong stages.
- Select Sofa and press F to focus; use T for texture and B for bump effects.
- Press V to access the mapping panel and reset the logical point for (657.5, 708.0).

6.5 Inputs and interface

Orbiting changes the camera viewpoint, scrolling changes magnification, left/right arrows rotate the chosen furniture, and sliders expose illumination/mapping parameters. The newer code also provides presentation/camera framing improvements, but the runtime screenshots in this report are from the earlier build. This distinction is essential for reproducible observations. The actual prototype does not access a phone camera, calculate AR anchors, or support tracked VR controllers.

7. PERFORMANCE, OPTIMIZATION AND LIMITATIONS

7.1 Practical rendering constraints

A desktop teaching application and a phone WebXR experience have different performance constraints. Responsive visual interaction requires consistent CPU simulation, GPU shading and frame presentation. Headset displays may require separate eye projections and particularly low motion-to-photon latency. Target frame budgets are design goals, not measurements taken from this Unity project.

Target rate	Theoretical frame budget
60 frames/s	1000/60 = 16.67 ms
72 frames/s	1000/72 = 13.89 ms
90 frames/s	1000/90 = 11.11 ms

7.2 Source-level optimization principles

- Low-complexity primitive geometry keeps the demonstration straightforward while allowing texture detail to dominate material appearance.
- Using albedo and normal textures avoids tessellating the sofa solely to depict fine pores.

- A simple custom forward shader supports explicit coefficients, but shader work and texture sampling still consume fragment budget.
- A single scene with controlled lighting makes A/B visual comparisons more reproducible than introducing many real-time light sources.
- Correct camera framing and responsive control placement improve usability without adding heavy scene geometry.

7.3 Memory and measurement integrity

Two 256 × 256 RGBA32 texture bases occupy 2 × 256 × 256 × 4 = 524,288 bytes, or 512 KiB, before mipmaps, padding or runtime driver overhead. This is an analytic estimate from the current source texture size, not a measured profiler reading. Model complexity, draw calls, shader variants and target hardware determine whether performance targets can actually be met.

7.4 Proposed validation

In Unity, use the Profiler/Stats window to record consistent frame time and draw-call measurements, and compare texture/bump toggles at fixed camera angles. For an eventual phone or headset deployment, test the actual devices and WebXR runtime, texture compression, occlusion and input latency. No frame-rate benchmark or mobile/headset execution is claimed in this submission.

8. APPLICATIONS AND DISCUSSION

8.1 Interpretation for the retail scenario

The laboratory demonstrates why appearance models matter beyond simple product geometry. A furniture preview can show the customer a coherent red chair and a material-rich sofa; in a real product, consistent environmental lighting would help the virtual item belong to the physical room. Fine material textures add useful visual information without the cost of high-resolution surface meshes. Screen-space mapping is essential when the customer taps the phone or selects an on-screen hotspot.

8.2 Benefits of a teaching-scale implementation

The Unity project is compact, easy to share as source, and uses no external artwork. Exposed controls help distinguish mathematical effects that are often hidden inside high-level render pipelines. Students can show the same object with changes in the Phong coefficients, compare albedo texture against normal mapping, and calculate a known viewport pixel

position. These properties support a short oral demonstration and transparent link to the three faculty questions.

8.3 Integration challenges

Real-world AR furniture placement would require a camera feed or XR compositor, physical-space scale calibration, robust plane detection, occlusion with room objects, consistent shadow estimation and a touch-to-ray interaction workflow. Product meshes would need verified dimensions, UV quality and textures from reliable sources. Mobile browser limitations include GPU budget, memory, device heat, permission prompts and screen diversity. The current demonstration intentionally does not claim to solve those deployment challenges.

8.4 Ethical and practical accuracy

A preview that exaggerates fabric texture, colour or size may lead to misleading purchasing decisions. Reliable retail visualization should be transparent about approximate lighting, simulated material parameters and device limits. This lab’s procedural leather and simplified red chair are classroom examples rather than measured fabric reproductions. The written report distinguishes code-supported functions, observed Play-mode evidence and proposed future capabilities.

8.5 PRACTICAL UNITY EVIDENCE — FULL ROOM

The following evidence consists of original student-provided screenshots taken in Unity 6000.3.0f1 Play mode on 18 October 2026. They were not generated as illustrations and were not reconstructed in a drawing tool. The screenshots depict the earlier working interface, before subsequent layout/presentation improvements were committed.

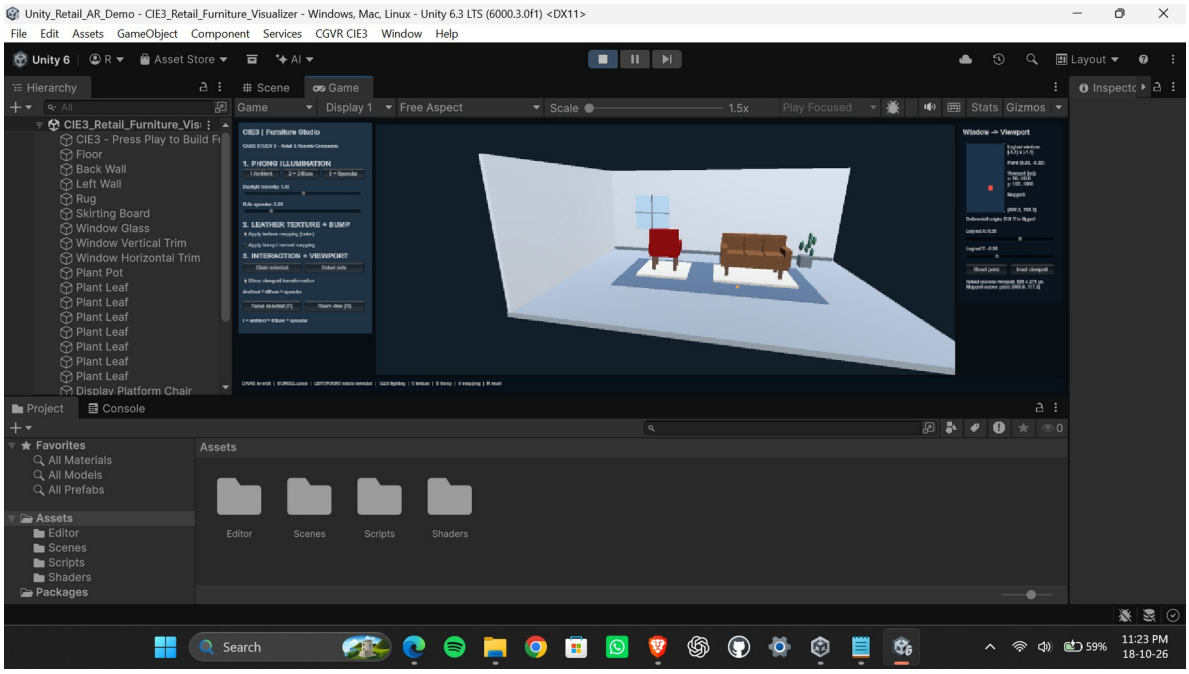


Figure 2. Genuine Unity Editor Game view (earlier working build): red chair, brown leather sofa, simulated room, and controls for lighting, material mapping and viewport transformation.

Observation: the wide-angle showroom includes both pieces of furniture, a window, floor, display platforms and separate UI regions. This directly substantiates that the earlier Unity visualizer rendered the 3D environment in Play mode. The view does not establish measured performance or whether individual keyboard toggles work in the latest code.

8.6 PRACTICAL UNITY EVIDENCE — CLOSE VIEW

The second capture uses a closer camera position and shows more clearly the red chair, sofa material and the transformation/illumination panel. These two views document camera interaction in an actual running Editor while also exposing presentation trade-offs.

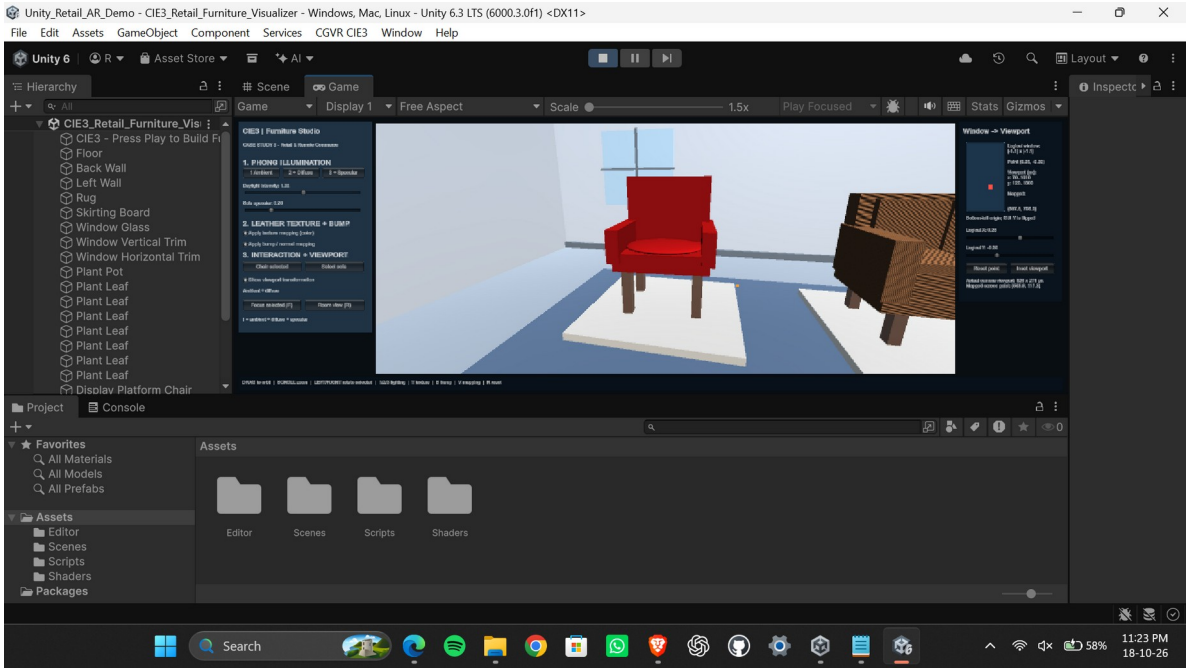


Figure 3. Genuine Unity Editor Play-mode close view from the earlier build; chair material, partially cropped leather sofa and mapping panel are visible.

Observation: the camera can move closer to the scene, but this older layout partly crops the sofa and its small text is harder to read inside an unmaximized Game tab. This explains why the later source introduced improved framing, larger panel text and a presentation mode. New Unity screenshots would be needed to claim those improvements were visually validated.

8.7 Evidence status

Verified by photographs: an earlier Unity Play-mode scene with two furniture objects and demonstration UI. Verified by source inspection: procedural material toggles, coordinate equations and later code changes. Not independently verified here: runtime state of the latest build, individual A/B lighting or bump captures, benchmark FPS, physical smartphone AR, headset operation or deployed WebXR. No output image is attributed to a test that was not run.

9. CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

This case study presents a realistic educational response to the RV University Retail and Remote Commerce scenario. A virtual red velvet chair motivates ambient, diffuse and specular calculations, which control how the surface responds to direction, light colour and viewer position. The brown sofa illustrates how UV colour textures and per-pixel normal perturbations contribute convincing material detail without increasing mesh polygon count. The window-to-viewport example derives the pixel mapping (0.25, −0.30) → (657.5, 708.0), showing how a normalized user-interaction space can be translated to a display region.

The source project integrates these techniques into a Unity showroom with procedurally built objects, a custom shader, controls and a reproducible saved scene. Actual screenshots confirm that an earlier build was operated in Unity Play mode. Its strength is educational clarity and the ability to relate code, equations and visible results; its limitations are simplified materials, unmeasured real-time performance and absence of live AR/VR device tracking. Thus the implemented work should be understood as a working desktop demonstration of rendering foundations, not as a deployed WebXR commerce platform.

9.2 Future scope

- Use physically based materials, realistic furniture meshes and measured/aligned textures for stronger visual fidelity.
- Estimate lighting from camera frames and add contact shadows, occlusion and correct scale in real rooms.
- Implement device-specific screen picking, tracked AR placement and VR stereo views with appropriate controls.
- Profile CPU/GPU frame times and memory on representative mobile devices; optimize geometry and texture compression.
- Collect comparable full-screen screenshots of each graphics toggle and validate the updated presentation version locally.

REFERENCES

[1] RV University, “CS4104 – Computer Graphics and Virtual Reality: Project / Case Study Assignment,” School of Computer Science and Engineering, faculty-supplied course document, 2026.

[2] M. Rishikesh, “Computer-Graphics-and-Virtual-Reality: CASE STUDY CIE3/Unity_Retail_AR_Demo,” GitHub source repository. [Online]. Available: https://github.com/Rishikeshsanin/Computer-Graphics-and-Virtual-Reality

[3] Unity Technologies, “Unity Manual and Scripting API,” documentation for Camera.rect, Camera.pixelRect, textures and ShaderLab. [Online]. Available: https://docs.unity3d.com/

[4] B. T. Phong, “Illumination for computer generated pictures,” Communications of the ACM, vol. 18, no. 6, pp. 311–317, Jun. 1975.

[5] T. Akenine-Möller, E. Haines, and N. Hoffman, Real-Time Rendering, 4th ed., CRC Press, 2018.

[6] J. F. Blinn, “Simulation of wrinkled surfaces,” SIGGRAPH Computer Graphics, vol. 12, no. 3, pp. 286–292, 1978.

APPENDIX A — SOURCE AND EXECUTION DETAILS

Unity version: 6000.3.0f1; rendering: Built-in Render Pipeline; programming: C# and ShaderLab/HLSL. The folder CASE STUDY CIE3/Unity_Retail_AR_Demo contains Assets, Packages and ProjectSettings. The saved scene is CIE3_Retail_Furniture_Visualizer.unity. The primary controller is RetailARCaseStudyDemo.cs and the shader is PhongFurniture.shader.

APPENDIX B — FACULTY CHECKLIST

- One case study chosen: Case Study 3 — Retail and Remote Commerce.
- All three scenario-specific assignment questions answered in Section 5.
- Mathematical derivation and substituted coordinates shown in Section 5.3.
- Architectural workflow, tables and captioned figures included.
- Frame-time budgets, limitations and optimization choices described.
- Original template cover and unsigned certificate format retained; all references numbered IEEE-style.